

Green Coding with Spring Boot

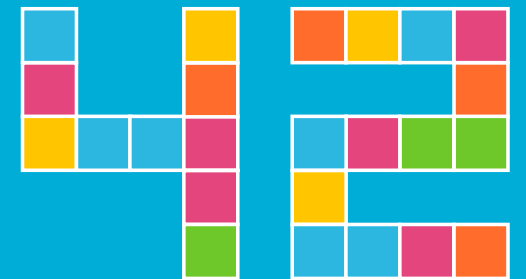
Sustainability as a Path to Better Software

Patrick Baumgartner

42talents GmbH, Zürich, Switzerland

@patbaumgartner

patrick.baumgartner@42talents.com



TALENTS

Abstract

Green Coding with Spring Boot: Sustainability as a Path to Better Software

Green coding is more than just a trend — it is a **promise of quality**.

Those who focus on **resource-efficient software** naturally make better choices: more efficient code, more stable systems, and lower operating costs. This session shows how to achieve that with **Spring Boot**.

We focus on **architectural principles** and **performance optimisations** that save energy — while also making the codebase **leaner, more maintainable, and more scalable**. This includes choosing the right **data structures and algorithms**, purposeful use of **Spring Boot components**, and optimising **servers and cloud infrastructures**.

For everyone who wants to develop not only in a *green* way — but above all in a **better** way.

Green Coding with Spring Boot

Sustainability as a Path to Better Code

The Code You Wrote Is Running Right Now

- Your ORM just fired **12 SQL queries** to render one page.
- Your test suite is rebuilding an unchanged module on every single push.
- Your pod idles at **3% CPU** — and it's still drawing power.
- **None of that was free.**

“The most sustainable code is the code you don’t execute.”

Who Am I?



Patrick Baumgartner

Technical Agile Coach @ **42talents**

Focus on **building of software solutions with and for humans**

Coaching, Architecture,
Development, Reviews, Training

Lecturer @ **Zurich University of Applied Sciences ZHAW**

Co-Organizer of **Voxxed Days Zurich**
and **JUG Switzerland, Java**

Champion, Oracle ACE Pro Java, ...

[@patbaumgartner](https://twitter.com/patbaumgartner)

Agenda

Agenda

 Why **green coding** matters

 Principles of **green software** & architecture

 **Measure first:** metrics, tools & demo (*Joular ecosystem*)

 **Automate:** CI/CD energy gates with greener - spring-boot

 **Optimise:** Spring Boot, JVM, Loom, Native Image

 **People & culture**

 **Takeaways**

Motivation

ICT's Carbon Footprint

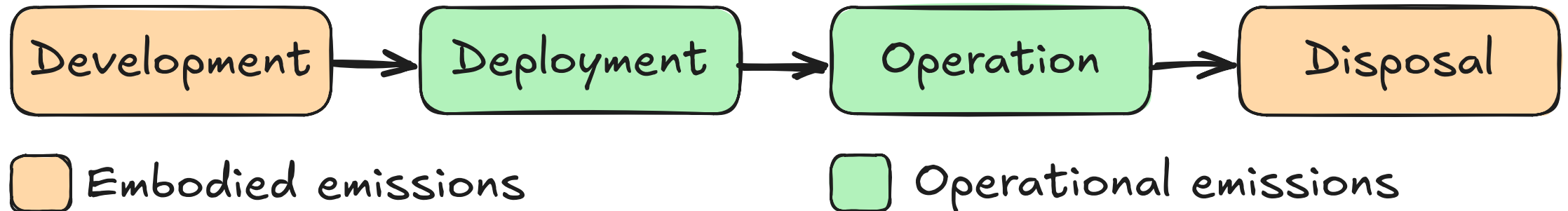
3–4% of global greenhouse gas emissions today — on par with **aviation** and **agriculture**

- Could reach **10–14% by 2040** if current growth continues
- Breakdown:
 - end-user devices **32–57%**
 - networks **22–25%**
 - data centres **18–41%**
- ~**25%** is **embodied** (hardware production) — before your first `git push`

Freitag et al. (2021) *The real climate and transformative impact of ICT*, Patterns (Cell Press) · Belkhir & Elmeligi (2018) *Assessing ICT global emissions footprint*, J. Cleaner Production

Where Do Software Emissions Come From?

- **Embodied**: hardware production, dev laptops, CI/CD infra
- **Operational**: data centres, networks, end-user devices
- Greener code cuts **operational emissions** and extends hardware lifetimes → lowers **embodied emissions**



Source: Freitag et al. (2021) *The real climate and transformative impact of ICT*, Patterns (Cell Press)

Your Build Pipeline Has a Carbon Cost

- **Embodied emissions** $\approx$ 25 % of ICT footprint (hardware you already own):
 - Developer laptops & workstations
 - CI/CD build servers & agents
 - Test & staging environments
 - Monitoring & observability systems
 - Cooling & power distribution
- Every failed test run, every unnecessary rebuild **burns energy**
- Extending hardware lifetimes **cuts embodied impact**

Run lean pipelines — optimised builds are greener builds

The Opportunity in Green Coding

- Inefficient code = **wasted CPU cycles**, higher bills, more emissions
- Green coding improves:
 - **Performance** → faster responses, better throughput
 - **Stability** → leaner, less complex systems
 - **Cost-efficiency** → lower compute & storage needs

Treat energy efficiency as a software quality attribute

Beware the Rebound Effect

- You make the app **10× faster** → users use it **10× more**
- New traffic **outgrows** the efficiency gain → net emissions rise
- Known in economics as the **Jevons paradox**

Defending against it:

- Track **energy per request / per user** — not just total
- Set **absolute energy budgets**, not only relative improvements
- Make efficiency gains **visible** in dashboards & reviews

Efficiency alone does not guarantee sustainability
→ **design for it deliberately**

Principles & Architecture

Core Principles of Green Software Architecture

- **Design lean systems** — avoid unnecessary layers
- **Efficient algorithms & data structures** — less CPU & memory
- **Right-size workloads** — no over-provisioning
- **Measure energy continuously**
- **Lifecycle view** — optimise runtime, maintainability, scalability

Every CPU cycle
and every byte transmitted
has a carbon cost ...

Architectural Trade-Offs

- **Monoliths**
 - Less orchestration, fewer network hops
 - Simpler deployment
- **Microservices**
 - Independent scaling, flexibility
 - Overhead from inter-service traffic, infra complexity

Rule of thumb: if not needed → keep it **modular monolith**

Stateless vs Stateful Architecture

- **Stateless**

- No session affinity → clean horizontal scaling
- Container-friendly, autoscales freely
- Less in-memory state → smaller JVM heap

- **Stateful**

- In-memory caches, WebSockets, sticky sessions
- Fewer DB round-trips — but ties instances together
- Harder to drain & restart idle instances

Rule of thumb: prefer **stateless** — idle instances cost nothing when scaled to zero

Measure First

Performance & Energy Metrics

- **Performance:** latency, throughput, response time
- **Energy:** joules per transaction, useful work per joule
- Faster $\neq$ always greener
- Tools:
 - **Joular Core** (system), **JoularJX** / **Joular Code - Java** (method level)
 - **JMeter**, **Gatling** for load
 - **JFR**, **async-profiler** for hotspots

Measure at Three Levels

Level	Focus	Tools
Code	Hot methods, allocations, CPU time	JFR, async-profiler, JoularJX
Application	Throughput, latency, energy / request	JMeter, Gatling
System	Host power, container energy, CO ₂	Joular Core, Kepler, Scaphandre

- Start at **application level** → identify regressions fast
- Drill to **code level** when you find a hotspot
- Validate at **system level** for absolute comparisons

Match the measurement level to your question

Software Carbon Intensity (SCI)

Defined by the **Green Software Foundation**:

$$SCI = (E \times I + M) / R$$

- **E** — energy consumed by the software (kWh)
- **I** — grid carbon intensity (gCO₂eq / kWh)
- **M** — embodied emissions (hardware share)
- **R** — functional unit: per request, per user, per hour

SCI makes optimisations **comparable over time** regardless of grid changes

See <https://sci.greensoftware.foundation/>



See <https://github.com/joular/joular>

The Joular Ecosystem

Tool	Role	Status
Joular Core	Universal system-level power monitor (Rust)	Beta v0.0.1
JoularJX	Java agent — per-method energy monitoring	Stable v3.1.0
Joular Code - Java	Next-gen Java agent — JoularJX successor	Alpha (Java 21+)

- **Joular Core** reads hardware power (RAPL, powermetrics, regression models)
- **JoularJX / Joular Code - Java** reads from Joular Core → attributes energy to **Java methods**

Joular ⚡ Core

Joular Core in Action

Start monitoring — **no instrumentation needed**, runs alongside any process:

```
# Monitor whole system
joularcore

# Attach to the Spring Boot JVM process (get PID: jps | grep petclinic)
joularcore --pid <PID> \
            --file joularcore-data.csv \
            --calibrate-cpu-idle-baseline
```

Sample CLI output:

```
[Joular Core] CPU: 17.42 W | GPU: 0.00 W | PID 34768 (java): 2.32 W
[Joular Core] CPU: 11.51 W | GPU: 0.00 W | PID 34768 (java): 1.25 W
```



See <https://github.com/joular/joularjx>

JoularJX in Action

Attach as Java agent — **no code changes** needed:

```
java -javaagent:joularjx-3.1.0.jar \  
-jar spring-petclinic.jar
```

config.properties:

```
filter-method-names=org.springframework.samples.petclinic  
save-runtime-data=true  
application-server=true      # required for Spring Boot  
joular-core=true             # use JoularCore as power source
```

Results in `joularjx-result/<pid>/`:

- `app/runtime/methods/` — energy per method per second (Joules)
- `app/total/methods/` — cumulative totals at shutdown

What JoularJX Found in spring-petclinic

Method	Energy (J)	Share
<code>OwnerController.findPaginatedForOwnersLastName</code>	0.81	53 %
<code>VetController.findPaginated</code>	0.26	17 %
<code>PetController.processCreationForm</code>	0.20	13 %
<code>PetTypeFormatter.parse</code>	0.08	5 %
<code>OwnerController.findOwner</code>	0.06	4 %

One method — `findPaginatedForOwnersLastName` — eats 53 % of business-method energy

Share of filtered petclinic-method energy over a 60 s load test (JVM startup/idle excluded).

`PetTypeFormatter.parse` : Spring's type Converter, invoked on every pet form submission.

Closing the Loop — After the Fix

Replace LIKE %name% (full table scan) with an **indexed prefix query**:

```
@Query("SELECT o FROM Owner o WHERE o.lastName LIKE :lastName%")
Page<Owner> findByLastNameStartingWith(
    @Param("lastName") String lastName, Pageable pageable);
```

Scenario (<code>findPaginatedForOwnersLastName</code>)	Energy (J)	Change
Before — LIKE %name% full table scan	0.81	—
After — indexed prefix search	~0.24	-71 %

Next: add `@Cacheable` to `VetController.findPaginated` — eliminates the 17 % allocation with one annotation.

Before value measured with JoularJX 3.1.0 on spring-petclinic. After (~0.24 J) is a projected value — indexed prefix query eliminates the full table scan. Values vary by hardware and load profile.

Joular ⚡ Code

See <https://github.com/joular/joularcoder-java>

Joular Code - Java ⚡ (Alpha)

Next-gen successor to JoularJX — Java 21+, requires Joular Core running:

```
# 1. Start Joular Core (ring buffer backend)
joularcore --app java --ringbuffer --silent

# 2. Attach agent – no code changes needed
java -javaagent:joularcodejava-0.0.1-alpha-4.jar \
-jar spring-petclinic.jar
```

joularcodejava.properties:

```
power-source-type=ringbuffer
methods-filtering-prefix=org.springframework.samples.petclinic
results-path=joular-code-java-results
```

Sample from methods-power-app.csv — **call-branch granularity** (peak W · cumulative J per 60 s test):

Greener Spring Boot

greener-spring-boot ⚡

- Maven & Gradle **build plugin** to measure Spring Boot energy consumption
- Wraps **Joular Core** (RAPL) + optional **JoularJX** (method-level)
- Compares against a **stored baseline** → fail build on regression
- Generates an **HTML report** with trend chart (last 100 runs)
- Works locally and in CI (GitHub Actions, GitLab, Jenkins)

<https://github.com/patbaumgartner/greener-spring-boot>

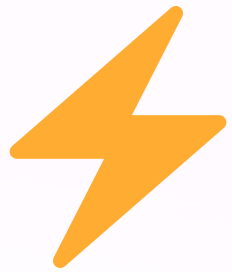
Plugin Quickstart

Maven — add to `pom.xml` and run `mvn greener:measure`:

```
<plugin>
  <groupId>com.patbaumgartner</groupId>
  <artifactId>greener-spring-boot-maven-plugin</artifactId>
  <version>0.2.0</version>
  <configuration>
    <externalTrainingCommand>
      oha -n 500 -c 10 ${APP_URL}/owners?lastName=
    </externalTrainingCommand>
    <threshold>10</threshold>
    <failOnRegression>true</failOnRegression>
  </configuration>
</plugin>
```

Gradle — add plugin and run `./gradlew measureEnergy`:

```
plugins { id("com.patbaumgartner.greener-spring-boot") version "0.2.0" }
greener {
  externalTrainingCommand.set("oha -n 500 -c 10 \${APP_URL}/owners?lastName=")
  failOnRegression.set(true)
}
```



LIVE DEMO

greener-spring-boot — mbarbieri

Energy as a CI Gate

- On every **PR**: measure energy, compare to `main` baseline
- Build **fails** if regression exceeds threshold (default 10 %)
- PR comment shows the delta automatically:

```
⚡ greener-spring-boot - Energy Report
-----
Baseline (main):  940.03 J
Current (PR):     920.49 J
Delta:            -2.09% ≈ UNCHANGED (threshold ±10%)
```

A PR introducing a regression (e.g. +11 %) prints ⚠ REGRESSION and fails the build automatically.

Shift "**measure before merge**" from aspiration to **automation**

Optimisations in Practice

Smarter Threads with Project Loom

- OS threads are **expensive** — they block on I/O and consume memory at scale
- **Virtual threads** (Java 21+) are lightweight and scheduler-managed by the JVM
- Better CPU utilisation, lower memory footprint, high concurrency without a reactive rewrite

Avoid **thread pinning** — synchronized blocks and blocking drivers stall carrier threads

Native Images & AOT with Spring Boot 4.x

- **GraalVM native image** — near-instant startup, minimal memory footprint
- Best for **serverless**, **scale-to-zero**, and **spiky loads**
- Trade-offs: longer build times, reflection hints needed, sometimes lower peak throughput

JVM for sustained throughput — **native** for cold starts and scale-to-zero

Spring Boot 4.x — Green Wins Out of the Box

- **Virtual threads** — if enabled; high concurrency without a reactive rewrite
- **AOT Cache** — pre-computed metadata cuts JIT warm-up energy on every restart
- **CRaC / Project Leyden** — checkpoint/restore: near-native cold starts without sacrificing peak throughput
- **Lean auto-configuration** — conditional wiring, unused beans never loaded
- **Actuator hooks** — ready integration points for greener-spring-boot

The greenest dependency is the one you never added — audit your starters regularly

Using Spring Boot Wisely

- **Persistence:** projections over full entities, `JOIN FETCH` to prevent N+1
- **Connection pool:** right-size HikariCP — oversized pools waste memory and connections
- **Caching:** Spring Cache on hot paths cuts repeated DB round-trips
- **Starters:** only pull in what you use — unused auto-configuration still has a cost
- **Profiling:** Hibernate Statistics + Actuator endpoints reveal query waste

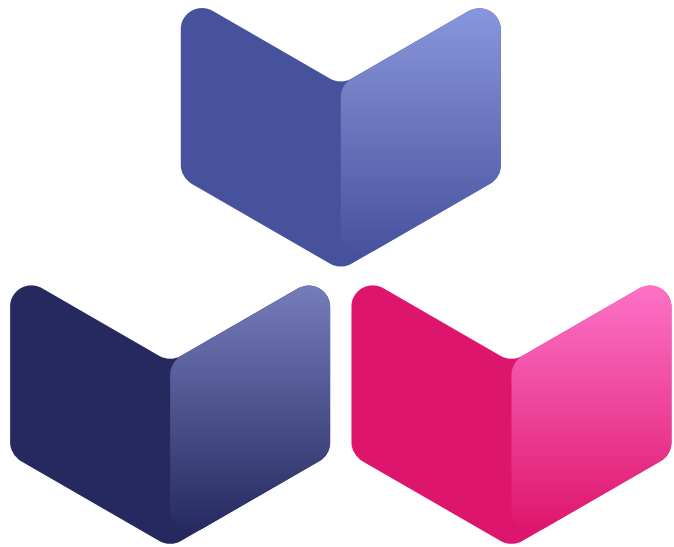
APIs with Less Overhead

- **Payload:** DTOs, pagination, server-side filtering — send only what is needed
- **Transport:** Gzip/Brotli + HTTP/2 reduce bytes on the wire
- **Caching:** response headers, CDN, and gateway cache keep responses close to clients
- **REST:** simple and cache-friendly — ETag and Cache-Control headers
- **GraphQL:** fewer roundtrips, but always use DataLoader to prevent N+1
- **Serialisation:** JSON for external APIs, **Protobuf / Avro** for internal services

Tune the JVM for Efficiency

- **Heap & GC:** right-size for your workload — G1 for throughput, ZGC for latency
- **Warm-up:** CDS / AppCDS or AOT cache cuts startup energy on every restart
- **Thread pools:** size to actual concurrency — oversized pools waste memory
- **Containers:** Java 21 respects cgroups automatically — no manual `-Xmx` guessing
- **Profiling:** JFR and flame graphs reveal real hotspots before you optimise

`spring.main.lazy-initialization=true` — skip beans until they are actually needed



Buildpacks.io

Friends, don't
let friends write
Dockerfiles!

@starbuxman



Infrastructure Choices Matter

- Grid carbon intensity **varies widely** by region and provider
- Favour data centres with low **PUE / WUE** (power and water usage effectiveness)
- Idle servers still draw power — autoscale before adding capacity
- Containers and serverless scale to zero for spiky workloads

Place workloads in **cleaner regions** — the grid mix matters

Lean Containers & Carbon-Aware Computing

- **Lean images:** Paketo **tiny** buildpack — no shell, fewer CVEs, faster cold starts
- **Layer order:** stable dependencies first, app code last — minimal rebuild times
- **Carbon-aware scheduling:** run batch jobs when renewable share is high
- **Region selection:** deploy to lower carbon-intensity locations when latency allows

Oversized images and unnecessary pulls have a carbon price too

Scale with Care

- *"Add more servers"* is rarely the answer — fix waste first
- Optimise queries, payloads, caches, and JVM before scaling out
- Right-size CPU and memory to actual load — avoid idle over-provisioning
- Autoscale on real thresholds, not assumed headroom

Spring Boot's fast startup = quicker, more efficient autoscaling

People & Impact

Developers Are the Multiplier

- You write code that runs **24/7 on servers you never see**
- A careless N+1 query, **multiplied by 1 million requests**, is not negligible
- You already fix **security bugs** — efficiency is the same discipline
- **Show the numbers**: energy dashboards change behaviour faster than policies
- Raise it in **code review**: “Does this query load more than needed?”

Shift from “*Does it work?*” to “*Does it work efficiently?*” — one question, infinite impact

The Industry Is Already Moving

- **Green Software Foundation SCI** — measurable carbon per unit of work, cited in enterprise procurement contracts
- **EU CSRD (2022/2464)** — large companies report Scope 3 digital emissions; applies from 2025
- **Cloud carbon intensity APIs** — Google Carbon-Free Energy, Azure Carbon, AWS Carbon Footprint: real-time data, available today
- **Net-zero supply chains** — Microsoft, Google, AWS public targets flow down to software vendors and their dependencies

Regulation measures the outcome — you control the code that produces it

How to Introduce This at Your Team

1. **Measure once, share openly** — run `greener-spring-boot`, post the baseline in your team channel
2. **Add it to Definition of Done** — *“Does this PR change energy consumption measurably?”*
3. **One question in code review** — *“Does this query, payload, or thread load more than needed?”*

Culture shifts when numbers are visible — one baseline, one dashboard, one conversation

Conclusion



Spring Boot



Spring Data



Spring for GraphQL



Spring Framework



Spring Modulith



What to Remember

- **Green coding = better coding** — same practices that save energy make code better
- Energy efficiency is a **software quality attribute**, not a compromise



Measure →



Optimise →



Repeat

Spring Boot 4.x + Java 21+ = the solid base you already have

Start Tomorrow

- ✅ Add greener-spring-boot — **measure a baseline today**
- 🔍 Attach JoularJX — find your **top 3 energy hotspots**
- 🔧 Fix biggest offenders: **queries, payloads, pools**
- ⚙️ Tune JVM: right-size heap, lazy init, virtual threads
- 🌍 Greener infra: **cleaner regions, autoscale, lean containers**

Treat energy efficiency as a quality gate — then automate it

Question

Every optimised query,
every leaner service,
every conscious design choice
moves us towards better
and greener software.

Green Coding with Spring Boot

Sustainability as a Path to Better Code

References

- Stocker, M. [Carbon Emissions in ICT](#)
- Stocker, M. [Evolution of Energy Usage in Spring Boot](#)
- Nouredine, A. [JoularJX](#)
- Nouredine, A. [Joular Core](#)
- Nouredine, A. [Joular Code - Java](#)
- Ruch, J. [Towards Greener Software](#)
- Zimmermann, O. [Growing Green Software](#)
- Zaragoza, T. et al. [A systematic mapping study on software-based feedback for energy consumption](#)
- Freitag, C. et al. [The real climate and transformative impact of ICT](#) (2021), Patterns (Cell Press)
- Belkhir, L. & Elmeligi, A. *Assessing ICT global emissions footprint* (2018), J. Cleaner Production
- Alcott, B. [Jevons' paradox](#) (2005), Ecological Economics
- Green Software Foundation [SCI Specification v1.0](#)
- EU [CSRD Directive 2022/2464](#)
- ...and many more!